2021 IEEE International Conference on Quantum Computing and Engineering (QCE) | 978-1-6654-1691-7/21/$31.00 ©2021 IEEE | DOI: 10.1109/QCE52317.2021.00075

# Quantum algorithms for Monte Carlo integration using pseudo-random numbers

Koichi Miyamoto
*Center for Quantum Information and Quantum Biology, Osaka University*
Toyonaka, Osaka, Japan
koichi.miyamoto@qiqb.osaka-u.ac.jp

***Abstract*—Quantum algorithms for Monte Carlo integration (QMCI), which are based on quantum amplitude estimation (QAE), provide quadratic speed-up compared with classical counterparts, and are therefore widely investigated, along with applications to industries such as finance. One typical feature of problems in finance is high dimensionality, or, in other words, the necessity to create numerous random numbers (RNs) to calculate the integrand. In such a situation, the original implementation of QMCI, in which states encoding the probability distributions of the RNs are generated on different quantum registers, requires a large number of qubits. In this poster, we propose an implementation of QMCI, in which pseudo-random numbers (PRNs) are sequentially generated on one register, and therefore the qubit number is tremendously reduced. Moreover, we show that, when the integrand has the form such that contributions from different RNs are separable into different terms, we can also achieve time complexity reduction with respect to the number of dimensions by combining the nested QAE and use of PRNs.**

***Index Terms*—quantum algorithm, Monte Carlo integration, financial engineering, pseudo-random number**

Monte Carlo integration (MCI) is a widely used numerical technique to calculate an expected value $E[F]$ of a function $F(X_1, ..., X_D)$ which depends on stochastic variables $X_1, ..., X_D$. Recently, quantum algorithms for MCI (QMCI) [3], [4], which are based on quantum amplitude estimation (QAE) [4], [5], are widely investigated. They can output an $\epsilon$-close estimation of $E[F]$ with high probability, making $O(\epsilon^{-1})$ calls to the oracle to calculate $F$, which denotes quadratic speed-up compared to classical counterparts with $O(\epsilon^{-2})$ queries.

One target of industrial application of QMCI is credit portfolio risk measurement in finance. Banks have credit portfolios, that is, collections of loans they issue, and suffer from the default risk of obligors. To monitor the risk, they calculate some risk measures such as the Value-at-Risk (VaR), which corresponds to the $\alpha$-th (say, 99th) percentile point of the loss caused by defaults, by MCI. More concretely, in the Merton model [6], which is widely used in many banks, the loss $L$ in a credit portfolio containing $N_{\mathrm{obl}}$ obligors is calculated as follows:

$$L = \sum_{i=1}^{N_{\mathrm{obl}}} E_i 1_{\beta_i x_0 + \sqrt{1-\beta_i^2} x_i < z_i},$$

where $x_0, x_1, ..., x_{N_{\mathrm{obl}}}$ are independent standard normal random numbers, $E_i$ is the exposure (that is, loan amount) of the $i$-th obligor, $\beta_i \in (0, 1)$ and $z_i \in \mathbb{R}$ are model parameters set for each obligor, and $1_C$ is the indicator function taking 1 if the condition $C$ holds and 0 otherwise. Then, VaR is given as $V_\alpha$ satisfying $E[1_{L>V_\alpha}] = 0.01\alpha$. Application of QMCI to this problem was first investigated in [7].

Typically, MCIs in finance are extremely high-dimensional, which means that a large number of random numbers (RNs) are generated to calculate the intagrand. For example, in credit portfolio risk measurement, the number of RNs is $N_{\mathrm{obl}} + 1$, which can be of order $10^6$ in a large bank. This can be problematic in the original implementation of QMCI, which we hereafter call the *register-per-RN* way. In this way, one quantum register is prepared for each RN and the superposition state encoding the probability distribution of the RN is generated on the register. Obviously, the required qubit number in this way can be very large in proportion to the number of RNs.

Motivated by this, we propose the *PRN-on-a-register* way for QMCI. A pseudo-random-number (PRN) sequence is a seemingly random but deterministic sequence generated by some recursion formula, and commonly used in classical MCIs. Starting from some initial value, we repeat updating the PRN on *one* register and using it to calculate a part of the integrand, and finally obtain a sample integrand value. Since only one register is required in this procedure regardless of the number of RNs, we can drastically reduce qubits compared with the register-per-RN way.

More concretely, the calculation procedure in the PRN-on-a-register way is as follows. We first choose some PRN sequence $\{z_i\}_{i=1,2,...}$, and implement its recursion formula and *jump* formula, which calculates $z_i$ for a specified $i$, as oracles $O_{\mathrm{up}} : |z_i\rangle \mapsto |z_{i+1}\rangle$ and $O_{\mathrm{jump}} : |i\rangle|0\rangle \mapsto |i\rangle|z_i\rangle$, respectively. Besides, we assume that $F$ can be sequentially calculated as $y_1 = f_1(X_1), y_2 = f_2(y_1, X_2), ..., F(X_1, ..., X_D) = f_D(y_{D-1}, X_D)$ by some function $f_1, ..., f_D$, which is applicable to credit portfolio risk measurement. Then, we implement the following operation:

$$|0\rangle|0\rangle|0\rangle|0\rangle \to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle|0\rangle|0\rangle|0\rangle$$

This poster presentation is supported by MEXT Quantum Leap Flagship Program (MEXT Q-LEAP) Grant Number JPMXS0120319794.

This presentation summarizes the following two works; [1] by K. Shiohara of Hokkaido University and KM, and [2] by K. Kaneko, N. Takeda, and K. Yoshino of Mizuho-DL Financial Technology Co., Ltd., and KM.



Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:08:58 UTC from IEEE Xplore. Restrictions apply.

$$
\begin{aligned}
&\to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle |z_1^{(i)}\rangle |0\rangle |0\rangle \\
&\to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle |z_1^{(i)}\rangle |f_1(z_1^{(i)})\rangle |0\rangle \\
&\to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle |z_2^{(i)}\rangle |f_1(z_1^{(i)})\rangle |0\rangle \\
&\to \ldots \\
&\to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle |z_D^{(i)}\rangle |F^{(i)}\rangle |0\rangle \\
&\to \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |iD+1\rangle |z_D^{(i)}\rangle |F^{(i)}\rangle \\
&\qquad \otimes \left(\sqrt{F^{(i)}}\,|1\rangle + \sqrt{1-F^{(i)}}\,|0\rangle\right).
\end{aligned}
$$

Here, $N$ is the number of samples, $z_j^{(i)} := z_{iD+j}$ with $i \in \{0, 1, ..., N-1\}$ and $j \in \{1, ..., D\}$, and $F^{(i)}$ is the $i$-th sample value of $F$, which we calculate using $z_1^{(i)}, ..., z_D^{(i)}$ on behalf of $X_1, ..., X_D$. In this flow, we first create the equiprobable superposition of initial values $z_1^{(0)}, ..., z_1^{(N-1)}$ by $O_{\mathrm{jump}}$, and, using each of them, calculate $f_1$. Then, we update the PRN sequence to $z_2^{(i)}$ by $O_{\mathrm{up}}$, and calculate $f_2$ using it. Repeating similar operations, we finally get $F^{(0)}, F^{(1)}, ..., F^{(N-1)}$ in superposition, and encode these into the amplitude of $|1\rangle$ on the ancilla qubit. The probability to observe 1 on the ancilla in the final state is equal to $\frac{1}{N}\sum_{i=0}^{N-1} F^{(i)}$, and we estimate this as an approximation of $E[F]$, using QAE.

In [1], we present more detailed discussions: the detailed implementation of the PRN-on-a-register way including a concrete PRN generator with circuit diagrams, comparison of the error bounds in the register-per-RN and the PRN-on-a-register ways, application to credit portfolio risk measurement, and so on[1].

In addition to qubit reduction, use of PRNs provides another benefit. That is, we can achieve time complexity reduction with respect to the number of dimensions $D$ in QMCI, if the integrand $F$ has the *separable* form, which is applicable to credit portfolio risk measurement:

$$
F(\epsilon_{\mathrm{com}}, \{\epsilon_i\}_{i=1,...,D}; \{\vec{c}_i\}_{i=1,...,D}) = g\left(\sum_{i=1}^{D} f(\epsilon_{\mathrm{com}}, \epsilon_i; \vec{c}_i)\right),
$$

Here, $X_1, ..., X_D$ are replaced with the *common stochastic variable* $\epsilon_{\mathrm{com}}$ and the *individual stochastic variables* $\epsilon_1, ..., \epsilon_D$, which means that the dimension of the integral is now $D+1$. Note that contributions from different RNs are decomposed into different terms, and $F$ is calculated by summing up these and operating some function $g$ to the sum. Each terms is written with some common function $f$, whose inputs are $\epsilon_{\mathrm{com}}$, one of $\{\epsilon_i\}_{i=1,...,D}$, and a vector of parameters $\vec{c}$ taking $\vec{c}_1, ..., \vec{c}_D$.

[1]Note that, in [1], the register-per-RN and the PRN-on-a-register ways are called the "previous" and "our" methods, respectively.

In this case, using a PRN sequence $\{z_i\}_{i=1,2,...}$, we can calculate $F^{(j)}$, the $j$-th sample value of $F$, as $F^{(j)} := g\left(DS^{(j)}\right)$, where $S^{(j)} := \frac{1}{D}\sum_{i=1}^{D} f^{(i,j)}$ and $f^{(i,j)} := f(z_{j(D+1)+1}, z_{j(D+1)+i+1}, \vec{c}_i)$ with $i \in \{1, ..., D\}, j \in \{0, 1, ..., N-1\}$, and take their average as an approximation of $E[F]$. We can of course calculate $S^{(j)}$ by naively iterating calls to the oracle $O_{\mathrm{up}}$ for PRN update and the oracle $O_f$ for $f$ calculation, but, using the jump oracle $O_{\mathrm{jump}}$, we can take another way. The point is that, using $O_{\mathrm{jump}}$ and $O_f$, we can construct the oracle $O_{\mathrm{term}} : |i\rangle |j\rangle |0\rangle \mapsto |i\rangle |j\rangle |f^{(i,j)}\rangle$, which calculates $f^{(i,j)}$ for the specified term index $i$ and the specified sample index $j$. Therefore, we can calculate the terms $f^{(i,j)}$ in quantum parallel, and sum them up by QAE. Since this is done in calculating $F^{(j)}$, and we also sum them up by QAE, the entire calculation is a nested QAE.

This way can actually reduce the query complexity. We can estimate $S^{(j)}$, and eventually $F^{(j)}$, with an error up to the tolerance $\epsilon$, making $O(\epsilon^{-1})$ calls to $O_{\mathrm{term}}$. On the other hand, we can calculate $S_j$ with no error by iterative uses of $O_f$ and $O_{\mathrm{up}}$, which results in $O(D)$ queries in total. Therefore, if $\epsilon^{-1} \lesssim D$, this QAE-based method may provide reduction of the number of iterative queries by a $O(\epsilon^{-1}/D)$ factor.

However, we should note that this does not immediately imply time complexity reduction, since the contents of the iterated operations are different in the two ways. Namely, the QAE-based method replaces PRN update in the naive iteration with PRN jump, which is typically more time-consuming. Assuming that the runtimes of update and jump are $T_{\mathrm{up}}$ and $T_{\mathrm{jump}}$ respectively, the time complexity reduction ratio by the QAE-based method is $O\left(T_{\mathrm{jump}}\epsilon^{-1}/T_{\mathrm{up}}D\right)$.

In [2], there are more detailed discussions such as more strict error estimation in the above QAE-based method. [2] also considers application of this method to credit portfolio risk measurement, presenting a setting in which time complexity reduction is possible.

For future work, it is meaningful to explore more use-cases of QMCI in which the use of PRNs is beneficial. Although applications to some problems in finance have been investigated [1], [2], [9], it will be interesting to expand the scope to other industrial and scientific fields, especially where speed-up of high-dimensional MCI is an issue.

## References


[1] K. Miyamoto and K. Shiohara, Phys. Rev. A 102, 022424 (2020)
[2] K. Kaneko et al., Quantum Inf. Process. 20, 185 (2021)
[3] A. Montanaro, Proc. Roy. Soc. Ser. A 471, 2181 (2015)
[4] Y. Suzuki et al., Quantum Inf. Process. 19, 75 (2020)
[5] G. Brassard et al., Contemporary Mathematics 305, 53 (2002)
[6] R. C. Merton, J. Finance 29, 449 (1974)
[7] D. J. Egger et al., IEEE Transactions on Computers (2020)
[8] L. Grover and T. Rudolph, arXiv:quant-ph/0208112 (2002)
[9] K. Kaneko et al., arXiv:2007.01467 (2020)




Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:08:58 UTC from IEEE Xplore. Restrictions apply.